Small Adaptive Systems: Negative and Mixed Results for Online Structural Adaptation, Predictive Model Banks and Learned Features in Compact Controllers

Author: Eric Lam (independent researcher, Toronto)

Abstract

We ask whether a compact learner can improve its control of an unfamiliar simulated system through its own experience while model size, state and compute stay fixed. We report four development studies. (1) A 32-unit recurrent controller with 1,060 registered learned parameters was trained offline by backpropagation through a training simulator, then run with local online updates to weights and to a bounded set of connections. Compared with a fixed-weight network of the same size, it showed no consistent tracking advantage; disabling its online updates hurt the trained model but a separately trained fixed network performed similarly. (2) A bank of four online linear predictors, one per candidate action delay, with no offline training, lowered mean tracking error relative to the fixed neural controller in three evaluation suites. It failed on longer action delays, and an information bonus did not clearly improve on the same bank without the bonus. (3) Four bounded learned nonlinear features, accepted by a prospective validation gate, lowered fresh-draw control error by 6.8% relative to a linear predictor, but did not clearly beat fixed random features or ungated learning, and they predicted worse on a shared probe. We conclude that none of these mechanisms showed a reproducible advantage over strong fixed or simpler controls at matched scale. The results are reported as development evidence only.

1. Introduction

The goal of this line of work is a practical learning mechanism that lets a small system acquire useful control from limited experience, on hardware as modest as a desktop or, eventually, a phone with about 4 GB of RAM. The immediate standard is reproducible learning under fixed resource limits rather than a general-intelligence claim. Resource limits in the project charter were set provisionally: at most one million learned persistent weights, at most 32 MiB of fast state and adaptation buffers, and a first-trial budget of one GPU-hour.

Three mechanisms were tested. The first is a shared update rule that changes recurrent connection strengths and bounded connectivity from local signals, a loosely related direction to differentiable plasticity (arXiv:1804.02464) and lifelong developmental programs (arXiv:2406.09787). The second is an explicit online predictive learner that keeps competing models with different action delays and plans with them, related to multiple-model adaptive control. The third is a bounded learned feature mechanism, related to Cascade-Correlation. The project's own documents state that none of these is claimed as an original invention; the question is whether each yields a measurable advantage against strong, simple alternatives at the same capacity.

2. Method

Simulated task. The controller receives two sensor values and a target and chooses two bounded actions. Neural pilots switch the dynamics from A to B and back to A in 32-step phases within 96-step episodes. Four development families are used: familiar linear dynamics, longer action delay, cross-channel coupling and nonlinear damping. Idea 03 uses a separate NumPy world with 192 interactions per episode and three effects absent from the learner's linear predictor: actuator dead zone, actuator saturation and state-dependent action gain.

Neural controller (Pilots 001–002). A 32-unit recurrent network with 1,060 registered learned parameters. Each unit has eight fixed candidate source units and four active incoming connections. Three modes share initialization, candidate sources, training episodes and update counts: fixed (no fast weight or routing updates), plastic (weight updates only) and rewire (both updates). Offline training runs 150 optimizer updates, each over 32 episodes of 96 interactions, giving 460,800 transitions per model. The deployed controller uses only observations and targets. Pilot 002 changes only the routing signal by normalizing it per unit; the rewire model is retrained with the same seeds and budget, and the fixed and plastic checkpoints are reused.

Online predictive bank (Idea 02). Four linear models, one per action delay from 0 to 3, each predict the two observations from the current observations, the delayed action and a constant. Model weights come from exponentially discounted prediction error (decay 0.8, softmax temperature 0.01, 2% uniform floor). Each model is fit by ridge regression to the latest 16 transitions. A planner scores 25 constant candidate actions over an 8-step horizon. The comparison includes a single all-lag model (22 fitted coefficients) and a version with an information bonus. The bank fits 40 coefficients online and uses no offline training.

Learned features (Idea 03). A ridge linear predictor over 10 scalar inputs is augmented with up to four tanh features. Every 16 observations, a candidate feature is proposed and optimized for 12 normalized gradient steps. The gated variant freezes a candidate and accepts it only if its accumulated error over the next eight observations falls by at least 5% and the mean absolute MSE improvement exceeds 1e-4. The main predictor has at most 74 scalars.

Baselines and controls. Zero action; proportional feedback; recursive least squares (RLS) identification with a horizon-six constant-action controller; a fixed neural controller (three saved seeds); a passive bank without the information bonus; a single all-lag model; a linear predictor; four fixed random features; and ungated learned features.

Seeds and protocol. Neural pilots use training seeds 11, 23 and 37. Idea 02 uses seed bases 781001 (original development), 881001 (fresh draws) and 981001 (random hidden-change timing). Idea 03 uses feature seeds 11, 23 and 37 with seed bases 1081001 (development) and 1181001 (fresh draws). Idea 02 and Idea 03 wrote protocols before their results and froze their choices. The neural pilots and the first experiment were not preregistered, and their documents state that no confirmatory claim is licensed.

3. Experiments and Results

3.1 Neural controller: tracking mean squared error

Lower is better; values average the three training seeds for neural methods.

Environment	Fixed	Plastic weights	Rewire 001	Rewire 002	RLS	Zero action
Familiar	0.1000	0.1003	0.1005	0.0990	0.1978	0.1653
Longer delay	0.1414	0.1425	0.1416	0.1412	0.2820	0.1755
Coupling	0.1418	0.1421	0.1416	0.1426	0.2663	0.1800
Nonlinear	0.1276	0.1277	0.1281	0.1285	0.2176	0.1718

Pilot 002 raises the average number of replaced connections per decision from about 0.02 to 4.25–4.50, but this does not produce a consistent tracking advantage. Its familiar-family change relative to fixed is approximately a 0.95% improvement in mean error, but one of three training seeds is worse. The coupling and nonlinear means are slightly worse.

On familiar environments, both sensors were within 0.1 of target on 9.8% of steps for fixed, 8.1% for Rewire 002 and 45.1% for RLS. RLS therefore tracks closely more often, while the neural controllers are more conservative and have lower overall error.

3.2 Ablation and state reset

Environment	Rewire 002 (normal)	Same weights, online updates disabled
Familiar	0.0990	0.1125
Longer delay	0.1412	0.1653
Coupling	0.1426	0.1662
Nonlinear	0.1285	0.1353

Disabling updates hurts the trained model in every family, so it has learned to depend on its update mechanism. A separately trained fixed network performs similarly to the normal model, so this dependence is not a demonstrated comparative benefit. Resetting internal state on return to A changed return-phase MSE from 0.1087 to 0.1081 (familiar), 0.1387 to 0.1402 (delay), 0.1446 to 0.1445 (coupling) and 0.1514 to 0.1459 (nonlinear). This is mixed and does not show reliable retention, because the reset also removes hidden state.

3.3 Online predictive bank (Idea 02)

Mean squared tracking error across all steps and both channels, with four families weighted equally.

Suite	Bank + information	Bank	Single model	Fixed neural	Original RLS
Original development	0.1050	0.1099	0.1139	0.1277	0.2409
Fresh draws	0.1124	0.1092	0.1184	0.1184	0.2006
Random timing	0.1017	0.1030	0.1005	0.1225	0.1817

Paired differences of bank with information minus fixed neural, with descriptive 95% bootstrap intervals over environments, were −0.0228 [−0.0359, −0.0099] (original), −0.0059 [−0.0202, +0.0079] (fresh) and −0.0208 [−0.0294, −0.0115] (random timing). The fresh-draw interval includes zero. The fixed neural

controller wins the longer-delay family in all three suites; on fresh draws the bank with information scores 0.1281 there, against 0.1137 for the fixed controller.

The information bonus changed actions relative to the passive bank on 40.3% of steps. Its aggregate difference from the passive bank was −0.0050 (original), +0.0033 (fresh) and −0.0013 (random timing), and every paired interval includes zero. The single model has the lowest aggregate error in random timing, so the bank is not universally superior.

3.4 Learned features (Idea 03)

Mean squared control error on fresh draws, equal weight per family.

Method	Fresh MSE	Development MSE
Linear	0.06024	0.06211
Fixed random features	0.05674	0.06092
Learned, ungated	0.05508	0.05425
Learned, gated	0.05616	0.06064

Gated learning is 6.8% below linear on fresh draws (paired difference −0.00408, interval [−0.00774, −0.00085]). Its differences from fixed random features (−0.00058, [−0.00783, +0.00881]) and from ungated learning (+0.00108, [−0.00419, +0.00712]) are uncertain. The development improvement over linear (−0.00147, [−0.00557, +0.00212]) is also uncertain. On fresh runs the gate accepted 280 of 1,541 candidates (18.2%), leaving 1.94 active features on average.

On a fixed-action prediction probe, where every model receives the same observations and actions, mean prediction MSE was 0.009901 for linear, 0.010496 for fixed random features, 0.010599 for ungated learning and 0.010032 for gated learning. The gated variant is worse than linear by +0.000131 (interval [+0.000046, +0.000221]), so the control gain is not explained by better prediction.

4. Discussion

The results are mixed in an informative way. Online structural adaptation in a recurrent network changed behavior, and trained models depended on their update mechanism. Yet at matched size it did not produce a consistent advantage over a fixed network, and a separately trained fixed network performed similarly to the adaptive model. This pattern points to the credit signal as a candidate cause: the prediction-error proxy did not identify which connection change helped, and the project's own decision was to stop after two bounded versions rather than scale.

Explicit online prediction produced the clearest control gain relative to the fixed neural controller across three suites, without offline training. Its failures are equally informative: it lost on longer action delays, and an information bonus based on parameter uncertainty did not clearly improve on the same bank without the bonus. The bank also did not uniformly beat a single all-lag model.

Learned features reduced control error relative to a linear predictor, but the prospective gate did not clearly beat fixed random features or ungated learning, and it predicted worse on a shared probe. Accepting a feature after eight observations is not sufficient evidence that it captures a real nonlinearity. Lower error and reaching the target within tolerance also diverged: RLS had the highest error but was within tolerance on far more familiar-environment steps. Neither metric is a settled success criterion, because reachability of the targets was not verified.

The main practical conclusion is negative. Under the constraints used here, none of the three mechanisms showed a reproducible advantage over strong fixed or simpler controls, and the project records these results without promotion.

5. Limitations

- All experiments are simulated. Phone performance, energy use and real-device usefulness were not measured, and reported latencies are short desktop measurements.
- The dynamics vocabulary, feature function class, action lags and stable-dynamics prior were supplied by the designer, and the evaluation families were built with knowledge of the original generators.
- Targets are not guaranteed reachable under actuator limits. Tolerance frequencies are descriptive, not task-success rates.
- Pilot 002 used development evaluation data that had already been inspected. The neural pilots and the first experiment were not preregistered.
- Bootstrap intervals resample environments within families. They omit training-seed and family-selection uncertainty and are not corrected for multiple comparisons. The neural pilots used three training seeds per mode.
- The state-reset check confounds removal of hidden state with removal of adaptive state, and its fixed phase timing and repeated targets confound any memory interpretation.
- The PyTorch development evaluation process peaked at 487.8 MiB; this is not the exported runtime, whose standalone NumPy process peaked at 30.7 MiB in the neural pilot and 30.9 MiB in Idea 02.

6. Conclusion

Three compact adaptive mechanisms were tested against fixed, simpler and strong classical controls on simulated control tasks. Online structural adaptation did not show a consistent advantage at fixed size. An online model bank beat a fixed neural controller across three suites but failed on longer delays, and its information bonus did not clearly help. Learned nonlinear features lowered error relative to linear prediction without clearly beating fixed random features or ungated learning. These are development results with the limitations above. They do not establish general learning ability, and the next step recorded in the project is a diagnostic that isolates feature learning on stationary nonlinear dynamics with independent prediction queries.

References

1. Differentiable plasticity. arXiv:1804.02464. https://arxiv.org/abs/1804.02464
2. Neural Developmental Programs. arXiv:2307.08197. https://arxiv.org/abs/2307.08197
3. Lifelong Neural Developmental Programs. arXiv:2406.09787. https://arxiv.org/abs/2406.09787
4. Adaptive control using multiple models, switching and tuning. DOI 10.1002/acs.740. https://onlinelibrary.wiley.com/doi/10.1002/acs.740 (author names are not given in the source documents)
5. Klenske and Hennig. Dual Control for Approximate Bayesian Reinforcement Learning. Journal of Machine Learning Research, vol. 17, paper 15-162. https://jmlr.org/papers/v17/15-162.html (publication year not stated in the source documents)
6. Fahlman, S. E. and Lebiere, C. Cascade-Correlation. https://www.cs.cmu.edu/afs/cs.cmu.edu/Web/People/sef/sefCascor.htm (publication year not stated in the source documents)